

Python

Collections



Conteneurs

.Structures de données

- Les tuples (tuple): une séquence ordonnée d'objets immuable (non modifiables)
- Le string (str) peut être vu comme une séquence ordonnée de caractères

.Collections

- Les listes (list): une séquence ordonnée d'objets modifiable
- Les dictionnaires (dict): un ensemble de paires clé-valeur
- Le set (set): un ensemble d'objets distincts



Les tuples

- Listes immutables (non modifiables), créées en utilisant des parenthèses (facultatives)

```
#L'utilisation de parenthèses permet de déclarer un tuple  
my_tuple = ("Rue de Fer 25, Namur", "villa", 2000000, 4, "immoweb")  
.
```

count(value) : Compte le nombre de fois qu'un élément spécifique apparaît dans le tuple.

```
my_tuple = (1, 2, 3, 2, 2)  
count_of_twos = my_tuple.count(2)  
print(count_of_twos) # Affiche 3
```

3

index(value) : Renvoie l'index de la première occurrence d'un élément spécifique dans le tuple. Si l'élément n'est pas trouvé, cela lèvera une exception ValueError.

```
my_tuple = (1, 2, 3)  
index_of_two = my_tuple.index(2)  
print(index_of_two) # Affiche 1
```

1



Usages des tuples

• Coordonnées de points, dates

```
point = (3, 4)  
date = (2024, 8, 26)
```

• Retourner plusieurs valeurs dans une fonction

```
def min_max(numbers):  
    return (min(numbers), max(numbers))
```

• Passage de paramètres

```
def process_data(data):  
    x, y, z = data  
    # Traitement des données  
  
process_data((1, 2, 3))
```

• Déstructuration; assignation multiple

```
x, y = (10, 20)
```



Parcours de tuples

- Boucle pour

```
coordinates = (3, 4, 5)

for coordinate in coordinates:
    print(coordinate)
```

- Déstructuration

```
point = (10, 20)

x, y = point
print(f'x: {x}, y: {y}')
```

- enumerate()

```
coordinates = (3, 4, 5)

for index, value in enumerate(coordinates):
    print(f'Index {index}: {value}')
```

Index 0: 3
Index 1: 4
Index 2: 5

- Compréhensions

```
numbers = (1, 2, 3, 4)

squared = tuple(x**2 for x in numbers)
print(squared)
```

(1, 4, 9, 16)

Nb: tuple() converti une liste en tuple



Les string

- Peuvent être lues comme des tableaux de caractères

```
address = "Rue de Bruxelles"

print("Premier élément:\n", address[0])
```

- Ne peuvent pas être modifiées directement

```
mystring = "1234"
#Attention, Les string sont non modifiables caractère par caractère
mystring[2] = '0' #Renvoie l'erreur 'str' object does not support item assignment
```



Les listes : séquences ordonnées modifiables

```
#Pour déclarer une liste d'éléments, on utilise des crochets []
```

```
my_list = [1, 3, 5, 7]
```

```
my_list
```

```
[1, 3, 5, 7]
```

```
# Une liste peut contenir différents types de données
```

```
my_list2 = [3.14, 42, my_list, True, 'Namur']
```

```
my_list2
```

```
[3.14, 42, [1, 3, 5, 7], True, 'Namur']
```

```
#La fonction len() permet de connaître la longueur d'une liste
```

```
len(my_list)
```

```
4
```



Accès à une ou plusieurs éléments de la liste

- L'opérateur [x] permet d'accéder aux éléments d'une liste (commence à 0)

[1, 3, 5, 7]

- Un indice négatif démarre de la fin de la liste

```
#Un indice négatif part de la fin  
my_list[-1], my_list[-2]
```

(7, 5)

- Le symbole ":" permet d'accéder à plusieurs éléments consécutifs (découpage / slicing)

```
#On peut accéder à plusieurs éléments d'une liste en utilisant un colon (:)  
print(my_list[1:3]) #permet d'accéder aux éléments entre les indices 1 et 3 (3 exclu)  
print(my_list[1:]) #permet d'accéder aux éléments à partir de l'indice 1 (jusqu'au dernier)  
print(my_list[:2]) #permet d'accéder aux éléments du premier jusqu'à l'indice 2 (2 exclu)  
print(my_list[:]) #permet d'accéder à tous les éléments  
print(my_list[1:4:2]) #permet d'accéder aux éléments de 1 à 4 (4 exclu) par pas de 2
```



Ajout d'éléments à la liste

- **A la fin de la liste : append()**

```
fruits = ['pomme', 'banane']  
fruits.append('orange')  
# Résultat : ['pomme', 'banane', 'orange']
```

- **A une position précise : insert(index, element)**

```
fruits.insert(1, 'kiwi')  
# Résultat : ['pomme', 'kiwi', 'banane', 'orange']
```

- **Concaténer 2 listes : extend(iterable)**

```
fruits.extend(['ananas', 'mangue'])  
# Résultat : ['pomme', 'kiwi', 'banane', 'orange', 'ananas', 'mangue']
```



Supprimer des éléments d'une liste

- **Le premier élément de la liste : remove(element)**

```
fruits.remove('kiwi')
```

```
# Résultat : ['pomme', 'banane', 'orange', 'ananas', 'mangue']
```

- **Récupère puis supprime l'élément à l'index i : pop(i)**

```
dernier = fruits.pop()
```

```
# Résultat : ['pomme', 'banane', 'orange', 'ananas']
```

```
deuxieme = fruits.pop(1)
```

```
# deuxieme = 'banane'
```

```
# Résultat : ['pomme', 'orange', 'ananas']
```

- **Supprimer plusieurs éléments : del**

```
del fruits[0:1]# Supprime une tranche
```

- **Vider la liste**

```
fruits.clear()# Résultat : []
```



Compréhension de liste

• On peut utiliser le format suivant :

`new_list = [instruction for item in list if condition]`

Que va renvoyer le calcul suivant ?

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = [x**2 for x in numbers]
print(squared_numbers)
```



```
[1, 4, 9, 16, 25]
```

```
my_list1 = [1,2,3,4]
my_list2 = [5,6,7,8]

#my_list1'*my_list2 (produit de Hadamard (ou produit élément par élément)):
matprod = [ [a*b for b in my_list2] for a in my_list1]
matprod
```



.Réponse

```
my_list1 = [1,2,3,4]
my_list2 = [5,6,7,8]

#my_list1'*my_list2 (produit de Hadamard (ou produit élément par élément)):
matprod = [ [a*b for b in my_list2] for a in my_list1]
matprod
```

```
[[5, 6, 7, 8], [10, 12, 14, 16], [15, 18, 21, 24], [20, 24, 28, 32]]
```



Les set

- Collections non ordonnées d'éléments uniques (ordre insertion non conservé)
- On utilise des accolades {} pour déclarer un set.
- On ajoute avec .add et on enlève avec .remove ou .discard

```
my_set = {2,1,1,2,1,3,1}  
my_set
```

```
{1, 2, 3}
```

```
my_set.add(4)  
my_set
```

```
{1, 2, 3, 4}
```

```
my_set.remove(3)  
print(my_set)  # Affiche {1, 2, 4}
```

```
{1, 2, 4}
```

```
my_set.discard(3) # Pas d'erreur, même si 3 n'est pas dans le set  
print(my_set)  # Affiche {1, 2, 4}
```



Les dictionnaires

- Collections de type paire clé-valeur, déclarées avec accolades {}.
- Objets mutables (on peut ajouter des clés et des valeurs après déclaration).

```
house1 = {"address": "Rue de Fer 25, 5000 Namur", "website": "immoweb", "price": 400000, "surface": 150}
```

```
house2 = {"address": "Rue de Bruxelles 42, 5000 Namur", "website": "immoweb", "price": 350000, "surface": 200}
```

```
house3 = {"address": "Porte de Namur 25, Bruxelles", "website": "immovlan", "price": 400000, "surface": 100}
```

#On peut modifier une valeur

```
house1["surface"] = 170
```

#On peut aussi ajouter une clé / valeur

```
house1["rooms"] = 4
```



Parcourir un dictionnaire

• On peut itérer sur les clés d'un dictionnaire

```
ages = {'Alice': 30, 'Bob': 25, 'Charlie': 35}
```

```
# Itération avec items()|  
for name, age in ages.items():  
    print(f"{name} is {age} years old.")
```

• On peut utiliser des compréhensions de dictionnaire (génère un nouveau dictionnaire)

```
{clé_expr: valeur_expr for clé, valeur in iterable if condition}
```

```
adults = {name: age for name, age in ages.items() if age > 25}  
print(adults) # Affiche {'Alice': 30, 'Charlie': 25}
```

• On peut aussi utiliser des compréhensions de liste (génère une nouvelle liste)

```
[expression for clé, valeur in iterable if condition]
```

```
adult_names = [name for name, age in ages.items() if age > 25]  
print(adult_names) # Affiche ['Alice', 'Charlie']
```



Récapitulatif

- [] pour les listes
- () pour les tuples
- {} pour les dictionnaires ou les sets

